

NOLOCK Strategy

#sql-server #performance #concurrency #isolation

What NOLOCK Does

With (NoLock) is a table hint that sets the transaction isolation level to READ UNCOMMITTED for that specific table reference. It tells SQL Server: “Don’t acquire shared locks when reading this table, and don’t respect exclusive locks held by other transactions.”

```
Select 0.OrderId, 0.Status  
From 0eOrder 0 With (NoLock)  
Where 0.PharmacyId = @PharmacyId
```

The result: reads never block on writes, and writes never block on reads. The query runs faster because it skips the locking overhead and never waits for concurrent transactions to complete.

The Tradeoff: Dirty Reads

Without shared locks, you can read:

1. Uncommitted data. A transaction has inserted or updated a row but hasn’t committed yet. If it rolls back, you read data that never officially existed.
2. Partially updated rows. A transaction is updating multiple columns, and you might see some columns with old values and others with new values.
3. Phantom rows. Rows appear or disappear between reads in the same query because concurrent inserts and deletes aren’t blocked.
4. Skipped or doubled rows. On a large scan, if a page split occurs during your read, you might skip rows that moved or read the same row twice.

When NOLOCK Is Appropriate

Read-Only UI Display Procedures

Stored procedures that populate UI grids, dashboards, or display lists are the ideal use case. The requirements:

- Stale data is acceptable. The user will refresh the screen momentarily.
- No business decisions are made on the exact values returned.
- No financial calculations depend on the data.
- The procedure does not modify data. It is purely a SELECT.

Most pharmacy and fulfillment display procs fit this profile. Showing the list of manual fill groups, the sort station status, or the image export queue: if a row is off by one transaction, the user won’t notice.

High-Contention Read Scenarios

When a table is under heavy write pressure and reads are timing out or deadlocking, NOLOCK eliminates the contention entirely. This is a pragmatic choice when the alternative is “no data at all.”

When NOLOCK Is NOT Appropriate

- Financial calculations: reconciliation, billing, inventory counts where every row matters.
- Data modification: never read with NOLOCK and then update or delete based on what you read.
- Audit queries: regulatory or compliance reporting needs consistent snapshots.

- Cross-table consistency: if you need rows from table A and table B to be from the same point in time, NOLOCK can give you A from time T1 and B from time T2.

For these scenarios, use SNAPSHOT isolation or READ COMMITTED SNAPSHOT (RCSI) at the database level.

Consistent Application

When using NOLOCK in a procedure, apply it consistently to every table reference. Mixing locked and unlocked reads creates a false sense of consistency. You're getting dirty reads from some tables and locked reads from others, which is worse than committing to one strategy.

```
-- CONSISTENT: all tables use NoLock
Select G.GroupNum, O.OrderId, P.PharmacyName
From OeGroup G With (NoLock)
Join OeOrder O With (NoLock) On O.GroupNum = G.GroupNum
Join EndPharmacy P With (NoLock) On P.Id = O.PharmacyId

-- INCONSISTENT: missed one table. P is locked, G and O are not.
Select G.GroupNum, O.OrderId, P.PharmacyName
From OeGroup G With (NoLock)
Join OeOrder O With (NoLock) On O.GroupNum = G.GroupNum
Join EndPharmacy P On P.Id = O.PharmacyId -- missing NoLock
```

The inconsistent version can deadlock if another transaction locks P and then tries to lock G or O.

Syntax Notes

```
-- Table hint syntax (preferred for granularity)
From TableName T With (NoLock)

-- Session-level equivalent (affects all reads in the session)
Set Transaction Isolation Level Read Uncommitted

-- Query-level equivalent
Select ... From TableName T
Option (Querytracoon 1224) -- don't use this; just use the hint
```

The Missing NOLOCK Audit

When reviewing a procedure for performance, check every table reference. A missing NOLOCK on one table in a display procedure means that one table can cause lock waits while every other table read runs free. It's a common oversight during maintenance, where someone adds a new JOIN but forgets the hint.

```
-- Quick audit: search for table aliases without NoLock
-- in a proc that uses NoLock everywhere else.
-- Look for "Join TableName Alias On" without "With (NoLock)" following.
```

Real-World Example

In `Isp_RxfGetListOfManualFillGroups` (v48 to v49), the `InvStationNdcAssoc` table was missing its NOLOCK hint in the inclusion-list subquery while every other table in the procedure had one. This was a consistency fix. The table is read-only in this context (checking which NDCs are assigned to a fill station), and all surrounding references used NOLOCK. The missing hint was likely an oversight during a previous maintenance pass.

Related Concepts

- **Query Store Triage**: NOLOCK doesn't appear in Query Store directly, but lock wait times inflate duration metrics.
- **LEFT JOIN OR Anti-Pattern**: long-running scans from this pattern are especially vulnerable to lock contention.